

UNIVERSITY
OF WARSAW

Deep Neural Networks - Lecture 2

Marcin Mucha

October 28, 2026





Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)"

Plan

UNIVERSITY OF WARSAW

Gradient Descent

Overfitting

Plan

UNIVERSITY OF WARSAW

Gradient Descent

Overfitting

Gradient Descent - Recap

(Batch) Gradient Descent works by making steps in the direction of the negative gradient of the loss function, e.g.

$$L(W, B) = \frac{1}{n} \sum_i L_i(W, B),$$

where $L_i(W, B)$ is the loss on the i-th example, and depends on W and B.

Gradient Descent - Recap

(Batch) Gradient Descent works by making steps in the direction of the negative gradient of the loss function, e.g.

$$L(W, B) = \frac{1}{n} \sum_i L_i(W, B),$$

where $L_i(W, B)$ is the loss on the i-th example, and depends on W and B.

Computing the gradient requires going over the whole dataset and is very expensive, especially nowadays when often dealing with massive datasets.

Stochastic Gradient Descent

UNIVERSITY OF WARSAW

Stochastic Gradient Descent is instead computing the gradient $\nabla L_i(W, B)$ of the single example loss $L_i(W, B)$ and using it as an estimate of $\nabla L(W, B)$.

Stochastic Gradient Descent

UNIVERSITY OF WARSAW

Stochastic Gradient Descent is instead computing the gradient $\nabla L_i(W, B)$ of the single example loss $L_i(W, B)$ and using it as an estimate of $\nabla L(W, B)$.

This is very fast, but also the gradients are very noisy. In practice, with small enough η this has all the good properties of GD and works faster.

Stochastic Gradient Descent

Stochastic Gradient Descent is instead computing the gradient $\nabla L_i(W, B)$ of the single example loss $L_i(W, B)$ and using it as an estimate of $\nabla L(W, B)$.

This is very fast, but also the gradients are very noisy. In practice, with small enough η this has all the good properties of GD and works faster.

Also allows for online-learning, in particular no need to keep train set in memory.

Mini-batch SGD

UNIVERSITY OF WARSAW

Mini-batch Gradient Descent is a mix of both approaches. Split the data into mini-batches of fixed size s. For each mini-batch S compute the gradient of $\frac{1}{s}\sum_{i\in S} L_i(W, B)$ and use this as an estimate of the gradient $\nabla L(W, B)$.

Mini-batch SGD

Mini-batch Gradient Descent is a mix of both approaches. Split the data into mini-batches of fixed size s. For each mini-batch S compute the gradient of $\frac{1}{s}\sum_{i\in S} L_i(W, B)$ and use this as an estimate of the gradient $\nabla L(W, B)$.

This estimate is less noisy, but takes longer to compute.

Mini-batch SGD

Mini-batch Gradient Descent is a mix of both approaches. Split the data into mini-batches of fixed size s. For each mini-batch S compute the gradient of $\frac{1}{s}\sum_{i\in S} L_i(W, B)$ and use this as an estimate of the gradient $\nabla L(W, B)$.

This estimate is less noisy, but takes longer to compute.

Important: Computing the gradient for a batch of size 100 is not 100 times slower than computing the single example gradient, it is much faster than that! But the larger the batch size the more linear this behaviour is.

Technical issues

Some important details:

- Before each run through the whole dataset (*epoch*), you want to shuffle the dataset (could be expensive).

Technical issues

Some important details:

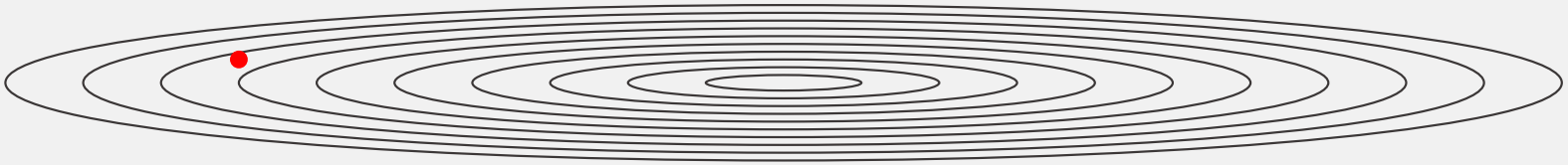
- Before each run through the whole dataset (*epoch*), you want to shuffle the dataset (could be expensive).
- If possible, you want all classes to be equally represented in batches. This is tricky if batches are small, or if classes are very unbalanced.

Technical issues

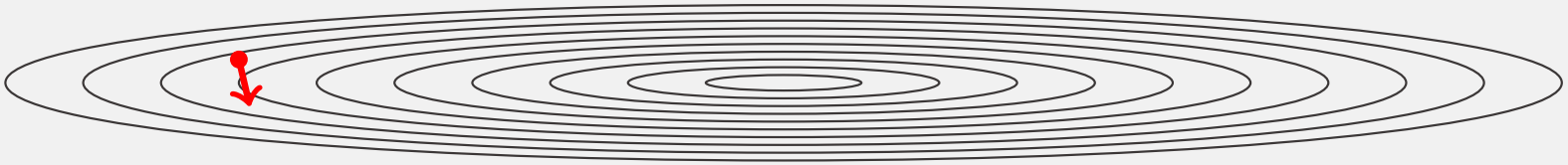
Some important details:

- Before each run through the whole dataset (*epoch*), you want to shuffle the dataset (could be expensive).
- If possible, you want all classes to be equally represented in batches. This is tricky if batches are small, or if classes are very unbalanced.
- The batch size does influence the learning rate, the nature of this relationship is not entirely clear, but experiments suggests that l.r. should be scaled linearly with batch size (up to a certain point, and perhaps not in the initial epochs).

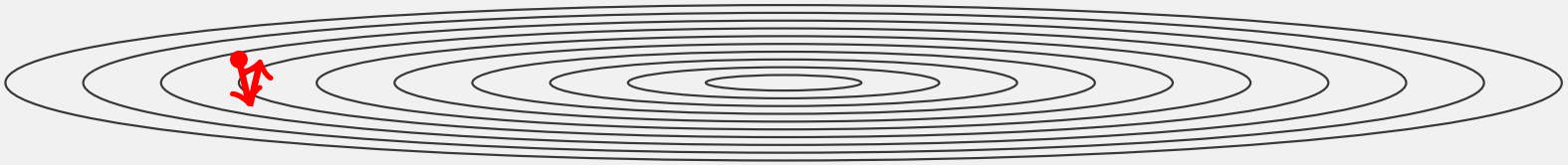
Problems with GD



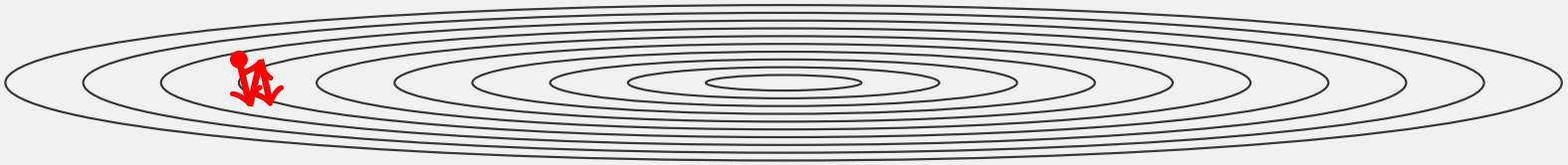
Problems with GD



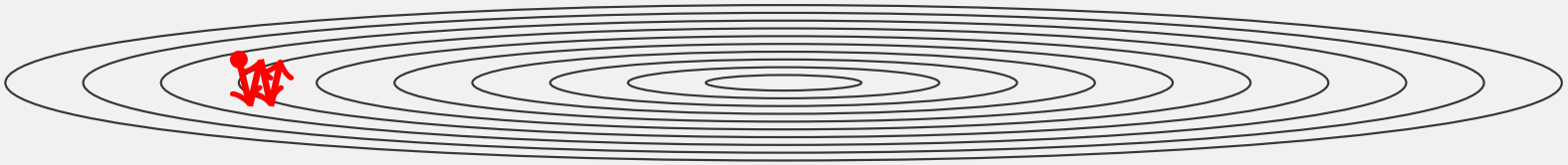
Problems with GD



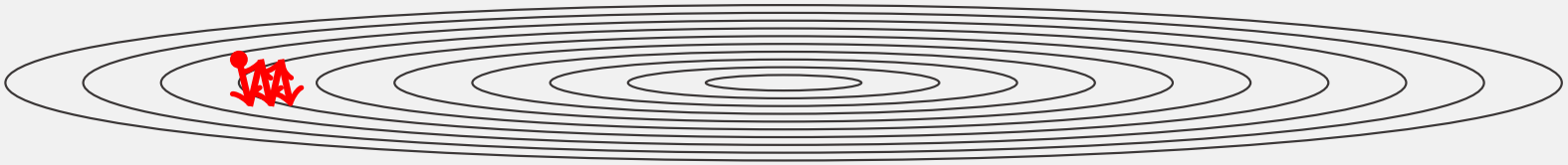
Problems with GD



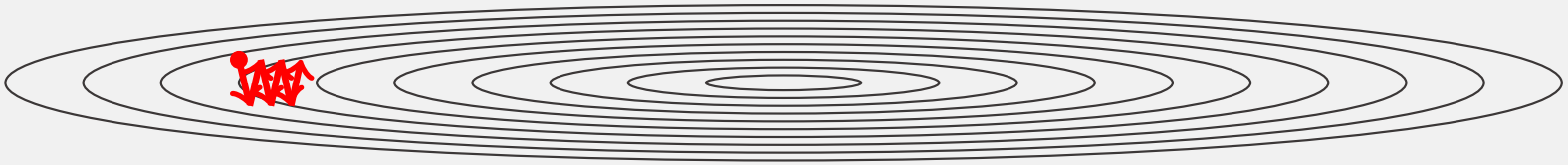
Problems with GD



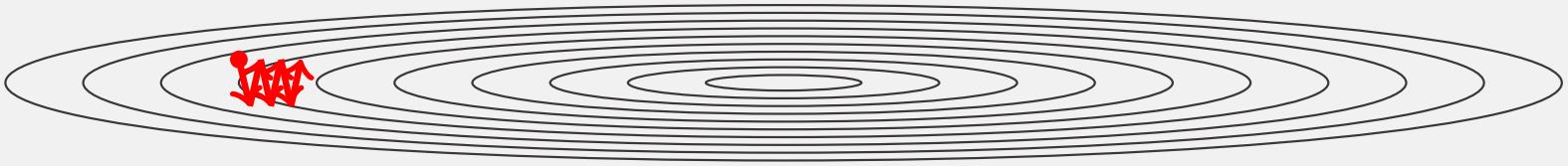
Problems with GD



Problems with GD

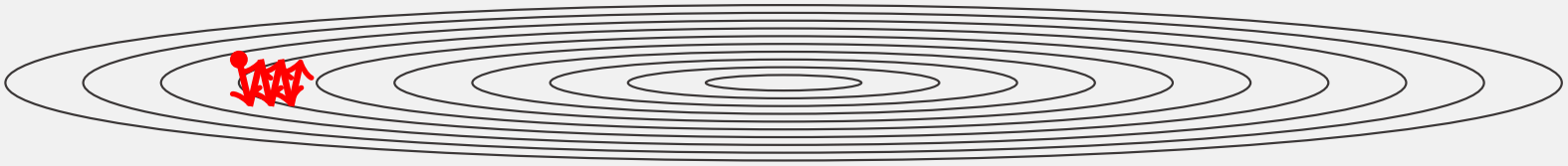


Problems with GD



You can try to avoid this kind of scenario (later).

Problems with GD



You can try to avoid this kind of scenario (later).

Or you can try to deal with it.

Momentum

Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.

Momentum

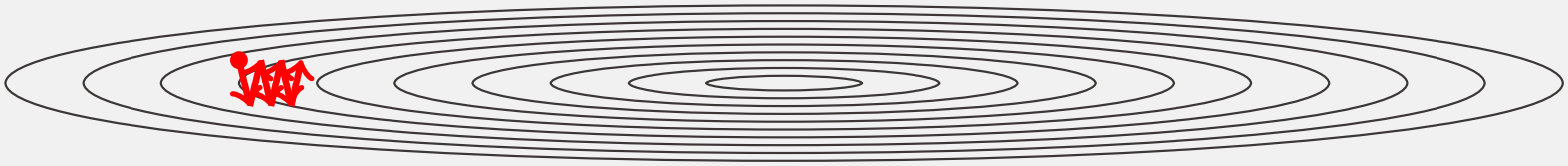
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Momentum

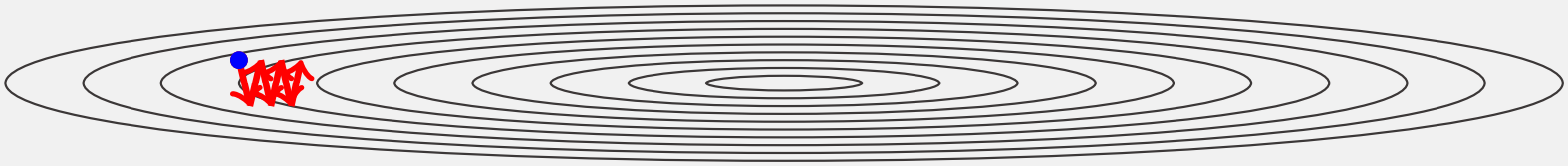
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Momentum

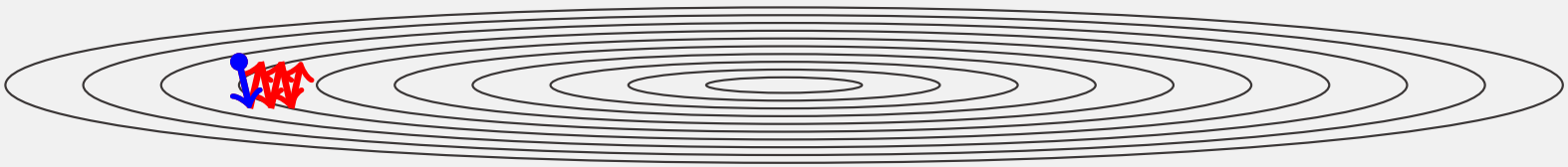
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Momentum

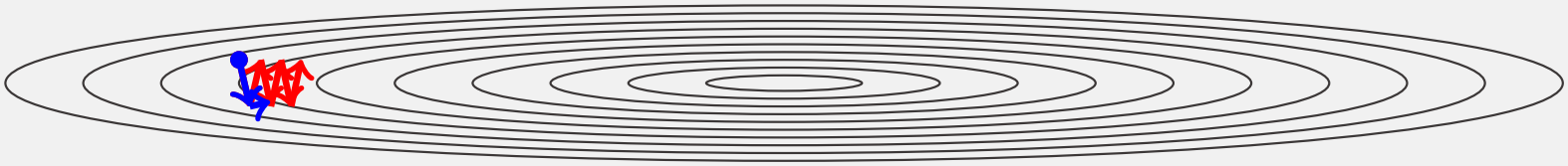
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Momentum

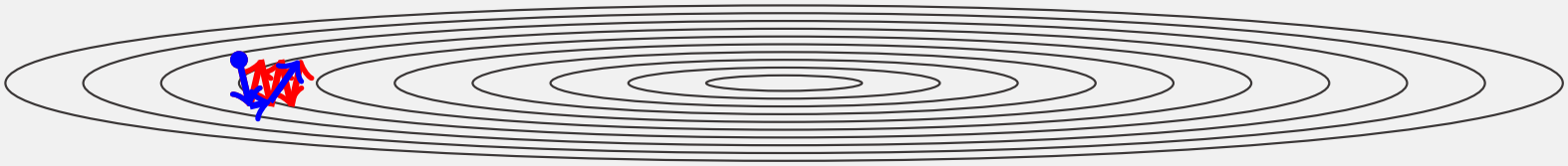
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Momentum

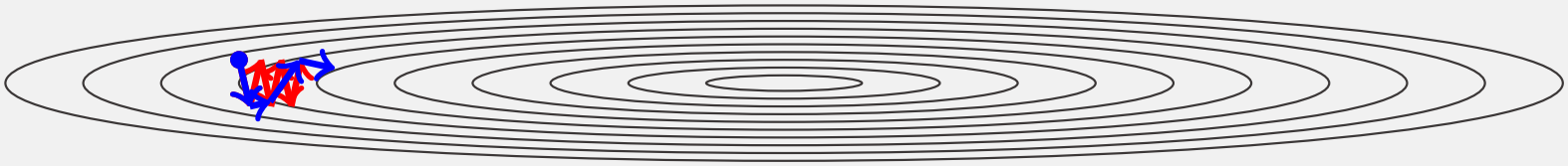
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Momentum

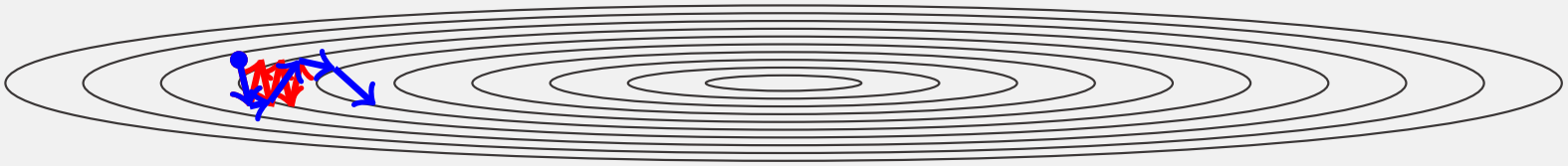
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Momentum

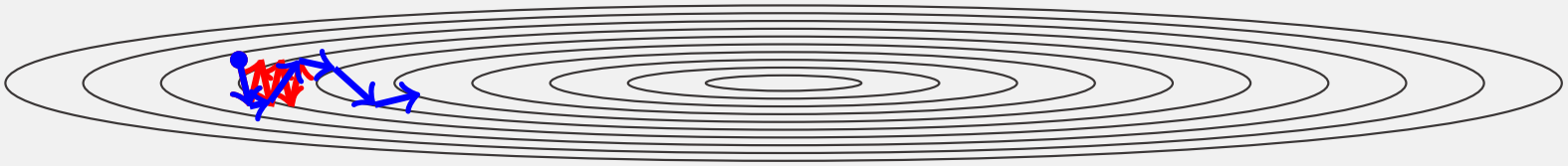
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Momentum

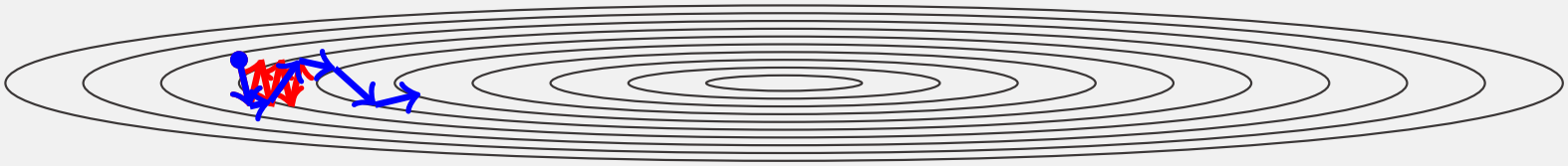
Idea: Accumulate gradients from recent iterations. Update parameters like this:

$$x_{t+1} = x_t + v_{t+1}$$

with

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

where γ is a parameter (momentum), e.g $\gamma = 0.9$. Start with a zero vector v_0.



Note: With momentum you might get different kind of oscillation, usually not a big problem.

Nesterov's momentum

This is the update vector again:

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

$$x_{t+1} = x_t + v_{t+1}.$$

Note that we will always move by γv_t and then some.

Nesterov's momentum

This is the update vector again:

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t),$$

$$x_{t+1} = x_t + v_{t+1}.$$

Note that we will always move by γv_t and then some.

Here is a different approach (*Nesterov's momentum*)

$$v_{t+1} = \gamma v_t - \eta \nabla L(x_t + \gamma v_t).$$

We move first to see what is there!

Nesterov's momentum

Might be useful to reformulate like this:

$$\tilde{x}_t = x_t + \gamma v_t,$$

Nesterov's momentum

Might be useful to reformulate like this:

$$\tilde{x}_t = x_t + \gamma v_t,$$

and then

$$v_{t+1} = \gamma v_t - \eta \nabla L(\tilde{x}_t),$$

Nesterov's momentum

Might be useful to reformulate like this:

$$\tilde{x}_t = x_t + \gamma v_t,$$

and then

$$v_{t+1} = \gamma v_t - \eta \nabla L(\tilde{x}_t),$$

and

$$\tilde{x}_{t+1} = x_{t+1} + \gamma v_{t+1} = x_t + v_{t+1} + \gamma v_{t+1} = \tilde{x}_t + (1+\gamma) v_{t+1} - \gamma v_t.$$

Nesterov's momentum

Might be useful to reformulate like this:

$$\tilde{x}_t = x_t + \gamma v_t,$$

and then

$$v_{t+1} = \gamma v_t - \eta \nabla L(\tilde{x}_t),$$

and

$$\tilde{x}_{t+1} = x_{t+1} + \gamma v_{t+1} = x_t + v_{t+1} + \gamma v_{t+1} = \tilde{x}_t + (1+\gamma) v_{t+1} - \gamma v_t.$$

Often works better.

Learning rate decay

Ideally, we want to use the largest η that does not lead to erratic behaviour.

Learning rate decay

Ideally, we want to use the largest η that does not lead to erratic behaviour.

When closer to optimum, might want to use smaller η to avoid overshooting, etc.

Learning rate decay

Ideally, we want to use the largest η that does not lead to erratic behaviour.

When closer to optimum, might want to use smaller η to avoid overshooting, etc.

Typically one uses an exponential schedule, e.g. decreasing the rate by a constant factor each epoch.

Learning rate decay

Ideally, we want to use the largest η that does not lead to erratic behaviour.

When closer to optimum, might want to use smaller η to avoid overshooting, etc.

Typically one uses an exponential schedule, e.g. decreasing the rate by a constant factor each epoch.

Other options: polynomial schedule, stages, cyclic learning rate. Additional parameters to tune.

Adagrad

Adagrad controls the learning rate for each parameter separately.

Adagrad

Adagrad controls the learning rate for each parameter separately.

Consider a single parameter w. The GD rule for w is

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}.$$

Adagrad

Adagrad controls the learning rate for each parameter separately.

Consider a single parameter w. The GD rule for w is

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}.$$

In Adagrad we use

$$w \leftarrow w - \frac{\eta}{\sqrt{G + \varepsilon}} \frac{\partial L}{\partial w},$$

where G is the sum of squares of all the gradients for w, and ε is just there to avoid division by zero, usually $\varepsilon = 10^{-8}$.

Adagrad

Adagrad controls the learning rate for each parameter separately.

Consider a single parameter w. The GD rule for w is

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}.$$

In Adagrad we use

$$w \leftarrow w - \frac{\eta}{\sqrt{G + \varepsilon}} \frac{\partial L}{\partial w},$$

where G is the sum of squares of all the gradients for w, and ε is just there to avoid division by zero, usually $\varepsilon = 10^{-8}$.

We do it for all parameters, by keeping G matrices and vectors on top of weight and bias matrices.

Adagrad

Advantages of Adagrad:

Adagrad

Advantages of Adagrad:

- Automatic learning rate decay.

Adagrad

Advantages of Adagrad:

- Automatic learning rate decay.
- Boosts rates of rare weights - useful for sparse learning.

Adagrad

Advantages of Adagrad:

- Automatic learning rate decay.
- Boosts rates of rare weights - useful for sparse learning.
- Quickly stabilizes fast moving parameters - controls oscillations.

Adagrad

Advantages of Adagrad:

- Automatic learning rate decay.
- Boosts rates of rare weights - useful for sparse learning.
- Quickly stabilizes fast moving parameters - controls oscillations.
- Boosts slow moving parameters - helps escape narrow ravines and saddles.

Adagrad

Advantages of Adagrad:

- Automatic learning rate decay.
- Boosts rates of rare weights - useful for sparse learning.
- Quickly stabilizes fast moving parameters - controls oscillations.
- Boosts slow moving parameters - helps escape narrow ravines and saddles.

Adagrad

Advantages of Adagrad:

- Automatic learning rate decay.
- Boosts rates of rare weights - useful for sparse learning.
- Quickly stabilizes fast moving parameters - controls oscillations.
- Boosts slow moving parameters - helps escape narrow ravines and saddles.

One problem: Sometimes the learning rates die to quickly.

RMSProp

RMSProp uses an exponential moving average of squares of past updates to help this.

$$G_t = \gamma G_{t-1} + (1-\gamma)\left(\frac{\partial L}{\partial w}\right)^2.$$

Typically, $\gamma = 0.9$.

RMSProp

RMSProp uses an exponential moving average of squares of past updates to help this.

$$G_t = \gamma G_{t-1} + (1-\gamma)\left(\frac{\partial L}{\partial w}\right)^2.$$

Typically, $\gamma = 0.9$.

Update rule is the same as in Adagrad.

RMSProp

RMSProp uses an exponential moving average of squares of past updates to help this.

$$G_t = \gamma G_{t-1} + (1-\gamma)\left(\frac{\partial L}{\partial w}\right)^2.$$

Typically, $\gamma = 0.9$.

Update rule is the same as in Adagrad.

Very efficient implementation of GD in practice.

Adam

Adam - a relatively recent (2015) algorithm that seems to beat all others in many applications.

Adam

Adam - a relatively recent (2015) algorithm that seems to beat all others in many applications.

It uses an EMA of squared gradients (like RMSProp) as well as EMA of gradients (like momentum methods).

Adam

Adam - a relatively recent (2015) algorithm that seems to beat all others in many applications.

It uses an EMA of squared gradients (like RMSProp) as well as EMA of gradients (like momentum methods).

It also uses special correction terms to fight initial bias towards zero of the EMAs.

Adam variants

Due to Adam's overwhelming success, many variants have been proposed - fixing deficiencies and offering specialized versions:

- AdamW - fixing L_2-regularization behaviour (see next slide)
- NAdam - Nesterov's momentum + Adam
- RAdam - *rectified* Adam, incorporates a kind of warm up schedule into Adam (fixing large variance in adaptive learning rates)

and many, many more...

AdamW

The equivalence between L_2 regularization and weight decay breaks down when using adaptive learning rate schedules. This algorithm attempts to fix this problem and make the choice of L_2 regularization parameter λ_2 indpendent of other choices, like learning rate.

AdamW

The equivalence between L_2 regularization and weight decay breaks down when using adaptive learning rate schedules. This algorithm attempts to fix this problem and make the choice of L_2 regularization parameter λ_2 indpendent of other choices, like learning rate.

Existing research seems to support the claim that this outperforms standard Adam, especially in terms of generalization.

See *Decoupled Weight Decay Regularization* Ilya Loshchilov, Frank Hutter

Which algorithm?

UNIVERSITY OF WARSAW

Vanilla GD and momentum methods require choosing the learning rate and learning rate decay. They also tend to give slower convergence, especially for sparse data.

Which algorithm?

UNIVERSITY OF WARSAW

Vanilla GD and momentum methods require choosing the learning rate and learning rate decay. They also tend to give slower convergence, especially for sparse data.

Adagrad based methods often work with less tuning, and in general better, but not always.

Plan

Gradient Descent

Overfitting

Motivation

UNIVERSITY OF WARSAW

Goal: We want our network to generalize - learn to recognize 6s and 9s and not *overfit* to the exact 6s and 9s from the training data.

Motivation

UNIVERSITY OF WARSAW

Goal: We want our network to generalize - learn to recognize 6s and 9s and not *overfit* to the exact 6s and 9s from the training data.

Exercise: Consider data with $N = 1000$ different examples in $x^i \in \{-1, 1\}^m$ and binary outputs $y^i \in \{0, 1\}$. Argue that a perceptron network with a single hidden layer of size N can learn to discriminate perfectly. Is this useful?

Motivation

UNIVERSITY OF WARSAW

Goal: We want our network to generalize - learn to recognize 6s and 9s and not *overfit* to the exact 6s and 9s from the training data.

Exercise: Consider data with $N = 1000$ different examples in $x^i \in \{-1, 1\}^m$ and binary outputs $y^i \in \{0, 1\}$. Argue that a perceptron network with a single hidden layer of size N can learn to discriminate perfectly. Is this useful?

Solution: The i-th perceptron in the hidden layer triggers only for x_i, by setting $w = x^i$ and $b = 1 - m$.

Motivation

UNIVERSITY OF WARSAW

Exercise: Can you simulate it using sigmoid units?

Motivation

UNIVERSITY OF WARSAW

Exercise: Can you simulate it using sigmoid units?

Solution: Setting the i-th perceptron to $\sum x_k^i x_k - m + 1$ gives 1 for x^i and at most -1 for other inputs. To get binary behaviour multiply weights and bias by a huge constant.

Motivation

UNIVERSITY OF WARSAW

Exercise: Can you simulate it using sigmoid units?

Solution: Setting the i-th perceptron to $\sum x_k^i x_k - m + 1$ gives 1 for x^i and at most -1 for other inputs. To get binary behaviour multiply weights and bias by a huge constant.

You can simulate any perceptron network this way. Bad, because you can achieve "this and nothing else" behaviour.

Motivation II

Another point of view on generalization: general concepts (6 vs a specific 6 from the input data) are resistant to noise.

How to create a network that does not change much when fed noisy data?

Motivation II

Another point of view on generalization: general concepts (6 vs a specific 6 from the input data) are resistant to noise.

How to create a network that does not change much when fed noisy data?

Use small weights!

L2 regularization

Standard way to avoid large weights is to include a penalty term in the objective function:

$$L(W, B) = \frac{1}{n} \sum L_i(W, B) + R_2(W, B)$$

where

$$R_2(W, B) = \frac{\lambda}{2} \sum_{w \in W} w^2.$$

Note: biases are not regularized!

L2 regularization

Standard way to avoid large weights is to include a penalty term in the objective function:

$$L(W, B) = \frac{1}{n} \sum L_i(W, B) + R_2(W, B)$$

where

$$R_2(W, B) = \frac{\lambda}{2} \sum_{w \in W} w^2.$$

Note: biases are not regularized!

How to implement this in GD?

L2 regularization - gradient

The gradient of the regularization part over the weights is independent from the mini-batch data and equals

$$\frac{\partial R_2}{\partial w} = \lambda w.$$

L2 regularization - gradient

The gradient of the regularization part over the weights is independent from the mini-batch data and equals

$$\frac{\partial R_2}{\partial w} = \lambda w.$$

In plain GD/SGD, multiply all weights by $(1 - \eta\lambda)$ before adding the data-loss update (computed at the original weights).

L2 regularization - gradient

The gradient of the regularization part over the weights is independent from the mini-batch data and equals

$$\frac{\partial R_2}{\partial w} = \lambda w.$$

In plain GD/SGD, multiply all weights by $(1 - \eta\lambda)$ before adding the data-loss update (computed at the original weights).

This is sometimes called *weight decay* for obvious reasons.

L1 regularization

Another idea - use L1 norm instead:

$$L(W, B) = \frac{1}{n} \sum L_i(W, B) + R_1(W, B)$$

where

$$R_1(W, B) = \lambda \sum_{w \in W} |w| .$$

L1 regularization

Another idea - use L1 norm instead:

$$L(W, B) = \frac{1}{n} \sum L_i(W, B) + R_1(W, B)$$

where

$$R_1(W, B) = \lambda \sum_{w \in W} |w| .$$

Exercise: Note that this does not really penalize large weights, but only their total! What is the effect on the learned weights?

L1 regularization

Another idea - use L1 norm instead:

$$L(W, B) = \frac{1}{n} \sum L_i(W, B) + R_1(W, B)$$

where

$$R_1(W, B) = \lambda \sum_{w \in W} |w| .$$

Exercise: Note that this does not really penalize large weights, but only their total! What is the effect on the learned weights?

Answer: L1 penalty promotes sparsity: it favors solutions with many zero weights, while some weights can remain large.

L1 regularization

UNIVERSITY OF WARSAW

Another idea - use L1 norm instead:

$$L(W, B) = \frac{1}{n} \sum L_i(W, B) + R_1(W, B)$$

where

$$R_1(W, B) = \lambda \sum_{w \in W} |w| .$$

Exercise: Note that this does not really penalize large weights, but only their total! What is the effect on the learned weights?

Answer: L1 penalty promotes sparsity: it favors solutions with many zero weights, while some weights can remain large.

This tends to avoid overfitting as well.

L1 regularization - GD

The gradient of the regularization part is again independent from the mini-batch data and equals

$$\frac{\partial R_1}{\partial w} = \lambda \text{sign}(w), \qquad w \neq 0.$$

L1 regularization - GD

The gradient of the regularization part is again independent from the mini-batch data and equals

$$\frac{\partial R_1}{\partial w} = \lambda \mathrm{sign}(w), \qquad w \neq 0.$$

The regularization contribution to each GD step is $-\eta\lambda\mathrm{sign}(w)$, directed towards zero.

L1 regularization - GD

The gradient of the regularization part is again independent from the mini-batch data and equals

$$\frac{\partial R_1}{\partial w} = \lambda \mathrm{sign}(w), \qquad w \neq 0.$$

The regularization contribution to each GD step is $-\eta\lambda\mathrm{sign}(w)$, directed towards zero.

For $w = 0$ the penalty is not differentiable. Ordinary subgradient steps can cross zero without landing on it, so they need not produce exact zeros.

Augmentation

The easiest way to avoid overfitting is to get more data - the model can no longer learn specific examples. What if there is no more data?

Augmentation

The easiest way to avoid overfitting is to get more data - the model can no longer learn specific examples. What if there is no more data?

Create synthetic data that *resembles the real data*. This is usually done by perturbing the real data - we make the network resistant to perturbations directly by adding perturbed inputs to the data!

Augmentation

The easiest way to avoid overfitting is to get more data - the model can no longer learn specific examples. What if there is no more data?

Create synthetic data that *resembles the real data*. This is usually done by perturbing the real data - we make the network resistant to perturbations directly by adding perturbed inputs to the data!

Exercise: Propose ways to create synthetic data for MNIST.

Augmentation

The easiest way to avoid overfitting is to get more data - the model can no longer learn specific examples. What if there is no more data?

Create synthetic data that *resembles the real data*. This is usually done by perturbing the real data - we make the network resistant to perturbations directly by adding perturbed inputs to the data!

Exercise: Propose ways to create synthetic data for MNIST.

Answer: There are many reasonable ideas: tiny rotations, shifts.

Dropout

UNIVERSITY OF WARSAW

Dropout: In each mini-batch step randomly remove a subset of neurons from the network, e.g. each with $p = \frac{1}{2}$ independently.

Dropout

UNIVERSITY OF WARSAW

Dropout: In each mini-batch step randomly remove a subset of neurons from the network, e.g. each with $p = \frac{1}{2}$ independently.

During prediction on test set, use all neurons. This generates f^l that are larger than when training so scale them by $1 - p$.

Dropout

UNIVERSITY OF WARSAW

Dropout: In each mini-batch step randomly remove a subset of neurons from the network, e.g. each with $p = \frac{1}{2}$ independently.

During prediction on test set, use all neurons. This generates f^l that are larger than when training so scale them by $1 - p$.

A generalization of this idea is to drop connections, not nodes - we will not discuss details.

Why dropout works?

UNIVERSITY OF WARSAW

- "This prevents complex co-adaptations in which a feature detector is only helpful in the context of several other specific feature detectors. Instead, each neuron learns to detect a feature that is generally helpful for producing the correct answer given the combinatorially large variety of internal contexts in which it must operate." – G. Hinton

Why dropout works?

UNIVERSITY OF WARSAW

- "This prevents complex co-adaptations in which a feature detector is only helpful in the context of several other specific feature detectors. Instead, each neuron learns to detect a feature that is generally helpful for producing the correct answer given the combinatorially large variety of internal contexts in which it must operate." – G. Hinton
- Averaging predictions over many networks is a good way to boost accuracy. Dropout trains many networks at the same time (sort of).

Why dropout works?

UNIVERSITY OF WARSAW

- "This prevents complex co-adaptations in which a feature detector is only helpful in the context of several other specific feature detectors. Instead, each neuron learns to detect a feature that is generally helpful for producing the correct answer given the combinatorially large variety of internal contexts in which it must operate." – G. Hinton
- Averaging predictions over many networks is a good way to boost accuracy. Dropout trains many networks at the same time (sort of).
- Dropout introduces noise, so features train to be more resistant to this kind of noise.

Why dropout works?

UNIVERSITY OF WARSAW

- "This prevents complex co-adaptations in which a feature detector is only helpful in the context of several other specific feature detectors. Instead, each neuron learns to detect a feature that is generally helpful for producing the correct answer given the combinatorially large variety of internal contexts in which it must operate." – G. Hinton
- Averaging predictions over many networks is a good way to boost accuracy. Dropout trains many networks at the same time (sort of).
- Dropout introduces noise, so features train to be more resistant to this kind of noise.
- Problem can have an obvious plateau of an easy solution. Dropout (especially on the input layer) can force the network to look beyond that.





Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)"